FIT3181/5215 Deep Learning — Weeks 7–11 cheat sheet. Green boxes = key concepts the quizzes test (likely to be re-used). **Red** = common trap. Shapes use PyTorch `batch_first=True`.

WEEK 7 · RNNs, LSTM, GRU (sequential data)

Probability of sequences

Chain rule: $p(x_{1:T}) = p(x_1)\prod_t p(x_t \mid x_{1:t-1})$

1st-order Markov: $p(x_{1:n}) = p(x_n|x_{n-1})\ldots p(x_2|x_1)p(x_1)$ (only previous item)

RNN = autoregressive model; $h_t = g(h_{t-1}, x_t)$ is a **lossy summary** of $x_{1:t}$. Pre-DL: HMM, Kalman filter, DBN, CRF.

Simple RNN (same U, W, V shared over time)

$h_0 = \tanh(x_0U + b)$ $h_t = \tanh(h_{t-1}W + x_tU + b)$
$\hat{y}_t = h_tV + c$ (regression) | $\mathrm{softmax}(h_tV + c)$ (classification)
Total loss $= \Sigma_t\, l(\hat{y}_t, y_t)$

U:[in, hid] W:[hid, hid] V:[hid, out]

Trap: x is always multiplied by **U**, h by **W**, output uses **V**. "h_1=tanh(h_0W+x_1**V**+b)" and "y=h**U**+c" are WRONG.

Parameter counting

RNN cell: in·h + h·h + h → in=10, h=5: 10×5 + 5×5 + 5 = 80
LSTM $= 4\times(\text{in}\cdot h + h^2 + h)$ GRU $= 3\times(\text{in}\cdot h + h^2 + h)$
(PyTorch keeps 2 bias vectors: +h per gate block)
Embedding = vocab × emb Linear = in·out + out

BPTT & vanishing gradient

$\partial L/\partial h_0 = \Sigma_t\, \partial \text{loss}_t/\partial\hat{y}_t \cdot \partial\hat{y}_t/\partial h_t \cdot \mathrm{diag}(1-\tilde{h}_t^2)W \cdots \mathrm{diag}(1-\tilde{h}_1^2)W$

Product of many W ⇒ **vanishing/exploding** gradients ⇒ RNN only captures **short-term** dependency. Fix = gating (add a linear path) → LSTM/GRU.
BPTT: unroll, forward & store, backprop from last step, **sum local gradients** per parameter, then SGD.

LSTM (Hochreiter & Schmidhuber '97)

$f_t = \sigma(x_tU_f + h_{t-1}W_f + b_f)$ $i_t = \sigma(\ldots_i)$ $o_t = \sigma(\ldots_o)$
$g_t = \tanh(x_tU + h_{t-1}W + b)$
$c_t = f_t \odot c_{t-1} + i_t \odot g_t$ (long-term) $h_t = o_t \odot \tanh(c_t)$ (short-term)

3 gates use **sigmoid** (0–1 proportion); candidate g uses tanh. 4 FC blocks.
Forget: proportion of long-term c_{t-1} carried forward. **Input**: how much of g_t stored. **Output**: proportion of current long-term c_t passed to new short-term h_t.

GRU (Cho et al. 2014, built for encoder–decoder)

$z_t = \sigma(x_tU_z + h_{t-1}W_z)$ $r_t = \sigma(x_tU_r + h_{t-1}W_r)$
$g_t = \tanh(x_tU_g + (r_t \odot h_{t-1})W_g)$ $h_t = (1-z_t) \odot h_{t-1} + z_t \odot g_t$

Merges c and h into one state; single **update gate z** = forget+input (z=1: open input / close forget); **no output gate**; **reset gate r** controls how much of the previous state is used. z, r → 1 ⇒ basic RNN.

Shapes (the most-asked quiz type)

Input ids [B, L] → Embedding [B, L, emb] → each RNN/GRU layer [B, L, *its own hidden*] → Flatten [B, L·h] → Linear [B, out]

`out, h_n = nn.GRU(...)(x)`: out = [B, L, h] (all steps); h_n = [layers, B, h]. Sequence-first form [L, B, in] ⇒ `torch.transpose(X,0,1)`.
Embedding lookup = one-hot $1_i \times E$ = row i of E.
Sentiment: (B,L)→(B,L,emb)→(B,L,128)→(B,128) last state→(B,1) sigmoid ≥0.5 pos.
Char-level text generation: x_0='*' start, '$' end; max log-lik = min Σ CE at each step; output = #chars neurons.

Topologies

many→one: sentiment analysis, sequence classification · **one→many**: image captioning · **many→many (enc–dec, delayed)**: machine translation · **many→many (synced)**: frame-level video classification.

★ Week 7 key concepts (what the quiz tests)

- **1st-order Markov**: each item depends only on the one before it: $p(x_n|x_{n-1})\ldots p(x_2|x_1)p(x_1)$.
- **Parameter count** = in×h (U) + h×h (W) + h (bias). Input 10, hidden 5 → 50 + 25 + 5 = 80.
- Input x is always multiplied by **U**, previous state h by **W**, output by **V**; the same U and W are reused at every time step.
- **Topologies**: sentiment = many→one · captioning = one→many · translation = many→many (encoder–decoder) · frame-level video = many→many (synced).
- **Shapes**: embedding gives [B, L, emb]; every RNN/GRU layer gives [B, L, *its own* hidden size] (L unchanged); Flatten gives [B, L×h] (5×16 = 80); Linear gives [B, out].
- `h, _ = gru(x)` keeps the **all-steps output** [B, L, h], not just the last state.
- All LSTM gates use **sigmoid**. Forget gate = share of long-term c_{t-1} kept. Output gate = share of the current long-term c_t passed into the new short-term h_t.

WEEK 8 · Word embedding, Word2Vec

Classical text features

- Pipeline: normalise → features (tf-idf, word2vec) → model (NB, SVM / LDA / k-means) → metrics (Acc, P/R; ROUGE, BLEU; NMI, Silhouette).
- Normalisation: expand contractions (isn't→is not), **lemmatisation** (cars→car, running→run, is→be), remove special chars, remove stop words.
- Bag-of-words: doc = Σ one-hot(words) = counts; stacked = term-by-document matrix.

$\mathrm{idf}(t,C) = \log(|C| / |C_t|)$ ($|C_t|$ = #docs containing t)
smoothed: $\log((1+|C|)/(1+|C_t|))$
$\mathrm{tfidf}(t,d,C) = \mathrm{tf}(t,d) \times \mathrm{idf}(t,C)$ L2-normalise: v / $\sqrt{(v_1^2+\ldots+v_n^2)}$
natural log: ln 2 = 0.69. Term in every doc ⇒ idf = 0. Ex: [0.69,0,0,0.69,0] → [0.71,0,0,0.71,0]

Representation learning

- Hand-crafted (SIFT/HOG/TF-IDF, extractor disconnected from classifier) vs DL (automatic, connected). Supervised rep = fits one downstream task; unsupervised = general.
- **Self-supervised**: invent a **pretext task** to turn raw data into supervised learning (e.g. predict rotation 0/90/180/270 → 4 labels). Word2Vec's pretext task: predict neighbours ("know a word by the company it keeps" – Firth).

Skip-gram (target → context)

$U \in \mathbb{R}^{N\times d}$ (input→hidden), $V \in \mathbb{R}^{d\times N}$ (hidden→output), N = vocab size, d = embedding size
$h = 1_tU = U_t$ (row t of U) $o = hV \in \mathbb{R}^{1\times N}$ $p = \mathrm{softmax}(o)$
$\log p(\text{cw=c} \mid \text{tw=t}) = U_tV_{:,c} - \log \Sigma_k \exp(U_tV_{:,k})$

$J(\theta) = \prod_{t=1..T} \prod_{-C\le j\le C,\, j\ne 0} p(w_{t+j} \mid w_t;\, \theta)$ → maximise; t = index of centre word

- Training pairs: (brown,the), (brown,quick), (brown,fox), (brown,jumps) for C=2.
- **Drawbacks**: softmax over huge N is expensive; probabilities tiny & hard to distinguish; depends on window size. Fixes: **hierarchical softmax** O(V)→O(log V), **negative sampling**.

Negative sampling

$p = \mathrm{sigmoid}(o)$ (binary, NOT a distribution) $\max \log p_c - \alpha \log p_{n1} - \alpha \log p_{n2}$

- Positive pair [(tw,cw),1] → maximise prob of being positive; random negatives [(tw,ng),0] → **minimise** prob of predicting them as positive.

CBOW (context → target)

$h = \frac{1}{4}(1_{c1}+1_{c2}+1_{c3}+1_{c4})U$ = average of context rows of U; $o = hV$; $p = \mathrm{softmax}(o)$
$J(\theta) = \prod_t p(w_t \mid w_{t-C},\ldots,w_{t-1},w_{t+1},\ldots,w_{t+C})$ window size = 2C+1

Using embeddings

Analogy: $\mathrm{argmin}_v \|v_{\text{Canberra}} - v_{\text{Australia}} + v_{\text{Paris}} - v\| = v_{\text{France}}$; cosine similarity for nearest words

- Init an RNN's embedding matrix with pretrained Word2Vec, then fine-tune (vs train from scratch).
- **doc2vec**: doc matrix D[N×p] + word matrix W[M×q], average/concat, predict word. **node2vec**: random walks = sentences. BFS = local/microscopic, DFS = global/macroscopic; p = return param, q = walk-away param; low p → BFS-like, low q → DFS-like. Also item2vec, med2vec, topic2vec…

Traps: vocabulary/dictionary size N = #unique words (10k), not #sentences. Embedding size d is a free hyper-parameter (NOT the window size). Skip-gram input = one-hot of the **target**; CBOW input = **average** of context one-hots.

★ Week 8 key concepts (what the quiz tests)

- **Skip-gram**: the centre word predicts each context word. **CBOW**: the context words predict the centre word. Window 7 = centre + 3 words each side.
- Analogy by vector arithmetic: king − prince ≈ queen − princess.
- One-hot = 1 at the word's index (check whether the dictionary starts at 0 or 1).
- Vocabulary size N = number of **unique words**, not sentences. Embedding size d is a chosen setting; the window size doesn't determine it.
- Skip-gram drawbacks: softmax over N is expensive; probabilities are tiny and hard to tell apart; results depend on the window size.
- **Negative sampling**: sigmoid output (outputs don't sum to 1); push (target, context) towards positive and (target, random word) away from positive.
- $\prod_t\prod_{j\ne 0} p(w_{t+j}|w_t)$ = skip-gram objective (t = centre word, maximise). $p(w_t|w_{t-C\ldots t+C})$ = CBOW objective, window = 2C+1.
- U is [N, d], V is [d, N]. Skip-gram: input = one-hot of the target, h = **that row of U**. CBOW: input = **average** of the context one-hots, h = average of their rows.
- Reading a toy network: vocab size = rows of U, embedding size = columns. A probability is the **softmax output** at the context word's index, not the raw score o = hV.

WEEK 9 · Seq2Seq, Attention, Transformer, BERT

Encoder–decoder / seq2seq

- **Encoder** encodes input sequence → context **c = h_T** (last **encoder** hidden state) summarising x. **Decoder** reads c → output sequence. In MT both are **RNNs**. Apps: MT, image captioning (CNN enc), code translation, QA.

$\max_\theta \Sigma_{(x,y)} \log P(y|x,\theta)$, $\theta = [\theta_e, \theta_d]$
$P(y|x) = P(y_{1:Ty}|c) \overset{(1)}{=} \prod_j P(y_j \mid y_{1:j-1}, c) \overset{(2)}{=} \prod_j P(y_j \mid q_{j-1}, c)$

- (1): c summarises $x_{1:Tx}$. (2): q_{j-1} summarises $y_{1:j-1}$. $P(y_j|q_{j-1},c)$ = dense layers on top of q_{j-1}, c (output = vocab+1).
- Variants: $q_{j-1} = f(q_{j-2}, y_{j-1})$ [Sutskever] or $f(q_{j-2}$, concat(c, $y_{j-1}))$ [Cho]; f = LSTM/GRU. Big drawback: c fixed across timesteps.
- **Inference**: phase 1 feed x to encoder to get c; phase 2 init decoder state with c (last encoder hidden), feed **<BOS>**, stop at <EOS>. **Greedy** = pick argmax each step (not sampling). **Beam search (k)**: keep top-k partial sequences by joint probability; finish when each beam outputs <EOS>; return best.

Attention (timely-varied context c_t)

- Fixed c is a bottleneck, overwhelmed by long inputs; c_t focuses on the input words most relevant to the current output word.

Global: $c_t = \Sigma_s\, a_t(s)\, \bar{h}_s$ over ALL encoder states; $a_t(s) = \mathrm{softmax}_s\, \mathrm{score}(q_t, \bar{h}_s)$
score: dot $q_t^T\bar{h}_s$ | general $q_t^TW_a\bar{h}_s$ | concat $v_a^T\tanh(W_a[q_t;\bar{h}_s])$
$\tilde{q}_t = \tanh(W_c[c_t; q_t])$ $p(y_t|y_{<t},x) = \mathrm{softmax}(W_s\tilde{q}_t)$

Local: window $[p_t-D, p_t+D]$ of encoder states; $p_t = S\cdot\mathrm{sigmoid}(v_p^T\tanh(W_pq_t)) \in [0,S]$
$a_t(s) = \mathrm{align}(q_t, \bar{h}_s) \cdot \exp(-(s-p_t)^2 / 2\sigma^2)$, $\sigma = D/2$

- Global = expensive for long sequences. Local = linear combination of encoder states **in a window**. Largest weight $a_t(s)$ = most important input word.

Transformer

- RNN problems: slow (sequential), poor long-range. Transformer: no recurrence/conv → **parallel**, great long-range; order recovered by positional encoding. GPT = Generative Pre-trained Transformer.

Self-attention

$Q = XW_Q$, $K = XW_K$, $V = XW_V$ (X: $[L, d_{\text{model}}]$, $d_Q = d_K$)
scores $B = QK^T/\sqrt{d_k}$ $[L,L]$ probs $A = \mathrm{softmax}(B, \mathrm{dim}=-1)$ $[L,L]$ $Z = AV$ $[L, d_V]$ $\mathrm{out} = ZW_O$ $[L, d_{\text{model}}]$

- $B_{ij} = Q_i\cdot K_j \approx \mathrm{sim}(x_i, x_j)$; Q, K are two other "views" of X. z_i depends mainly on x_i but every x_j contributes; **more similar x_j → more contribution**. Cost $O(L^2)$.
- **Multi-head**: each head has its **own** W_Q, W_K, W_V, computed independently in parallel; **concat heads → extra linear W_O**.
- **Masked SA** (decoder is causal): B.masked_fill(mask==0, −inf) on future positions → softmax gives 0 there.
- **Cross-attention**: Q = Y W_Q from **decoder**; K, V from **encoder output** X. B: [B, tgt_len, src_len]. Injects encoder output into decoder; part of decoder output only.

WEEK 9 (cont.) · Transformer blocks, BERT

Other blocks

$PE(pos,2i) = \sin(pos/10000^{2i/d})$ $PE(pos,2i+1) = \cos(pos/10000^{2i/d})$ → added to token embeddings

$LayerNorm(Y) = \gamma\,(Y - mean)/std + \beta$, stats over embedding dim (d_{model}) of each token

Pointwise FFN: $\bar{Z} = ReLU(ZW_1 + b_1)W_2 + b_2$ (same FFN per token)

Encoder block: MHSA → Add&LayerNorm (residual) → FFN → Add&LayerNorm; shape [B, L, d_{model}] kept, ×n blocks. Decoder: masked MHSA → cross-attn → FFN, then Linear+softmax.

LN has learnable γ, β; better than BatchNorm for variable-length sequences. PE encodes position of a **token in a sentence** (not sentence in batch); it is not the embedding itself.

BERT (encoder-only, pretrain + fine-tune)

Pretrain 1: **masked LM** – 15% of tokens chosen: 80% → [MASK], 10% → random word, 10% unchanged. Pretrain 2: **next sentence prediction** – 50% IsNext / 50% NotNext, predicted from [CLS] vector C.

Fine-tune **all** params. Classification: P = softmax(WC), $W \in \mathbb{R}^{M\times h}$. Span QA: $P_i = \exp(S^T T_i)/\Sigma_j \exp(S^T T_j)$, learn start S and end E vectors. Works because: Transformer + huge data + many params.

★ Week 9 key concepts (what the quiz tests)

- The **encoder** turns the input into context c (a summary of the input); the **decoder** reads c to produce the output. In MT both are RNNs and c = **last encoder hidden state**.
- In the factorisation, c summarises $x_{1:Tx}$ and q_{j-1} summarises $y_{1:j-1}$; $P(y_j|q_{j-1}, c)$ = dense layers on top of them. Feeding c only at the start or at every step are both valid.
- **Decoding**: encode to get c → start the decoder from the last encoder hidden state → first input <BOS> → greedy = take the argmax at each step (no sampling).
- A time-varying context beats a fixed one: it copes with long inputs and focuses on the input words relevant to the current output.
- **Global** attention = weighted sum of **all encoder** states; **local** = weighted sum of the encoder states inside a window. Never decoder states.
- $c_t = \Sigma\, a_t(s)\, \bar{h}_s$; the largest weight marks the most important input word.
- Positional encoding = position of a token **in the sentence**, **added** to the embeddings. LayerNorm normalises over d_{model}, has learnable γ, β, and beats BatchNorm on sequences.
- Self-attention: z_i comes mostly from x_i but every x_j contributes; more similar x_j gets more weight. Scores B come from Q and K, shape [L, L]; A = softmax(B); Z = **AV** (not BV).
- Multi-head attention: each head has its **own** W_Q, W_K, W_V and runs independently; heads are concatenated, then passed through W_O.
- Cross-attention: Q from the **decoder**, K and V from the **encoder output**; it feeds encoder information into the decoder (decoder side only).

WEEK 10 · Vision Transformers, Swin, Fine-tuning

Self- vs cross-attention

Self: source = target; Q, K, V from the **same (source)** sequence; captures **intra**-sequence dependencies. **Cross**: Q from **target**, K & V from **source**; **inter**-sequence. **Both compute token embeddings in parallel.** Attention ≈ soft lookup table (query soft-matches keys, returns weighted values).

Vision Transformer (ViT, "an image is worth 16×16 words")

224×224 image, 16×16 patches → $(224/16)^2 = 14\times14 =$ 196 tokens (+1 class token = 197)

flattened patch = 16·16·3 = 768 → linear projection → token embedding. Pixels as tokens: 224^2 tokens → 224^4 ops, impossible.

Token/visual word = **patch**. Prepend a **learnable** class token; MLP head on the class token at the **final** layer. Global info comes from **multi-head self-attention**; class token at final layer summarises all patch tokens.

ViT: naturally global, finds long-range dependencies between patches, needs **massive data** (ImageNet-21K, JFT-300M) – keeps improving with data, while CNN plateaus. More robust than CNNs to occlusion, distribution shift, adversarial perturbation, patch permutation. Low layers: local+global heads; high layers: global.

- CNN: inductive biases (locality, translation invariance) → OK on small data, but lacks global understanding.

CNN drawbacks: can't capture global info; can't capture spatial relationships of local objects; locality-sensitive. "Captures local info" and "combines local patterns" are strengths, not drawbacks.

Swin Transformer (hierarchical, shifted windows)

3×H×W → patch partition 4×4 → 48×H/4×W/4 → linear embed C×H/4×W/4 → 2C×H/8×W/8 → 4C×H/16×W/16 → 8C×H/32×W/32 (blocks ×2, ×2, ×6, ×2)

Patch merging: concat 2×2 neighbouring patches (4C) → linear → 2C: [C, H/4, W/4] → **[2C, H/8, W/8]** (spatial ÷2, depth ×2)

- **W-MSA**: split tokens into local windows (e.g. 4×4 patches), self-attention **inside each window independently**; same output shape; faster than global SA; but no cross-window communication.
- **SW-MSA**: shift windows right & down → tokens interact across windows; same output shape. Swin block = W-MSA block then SW-MSA block (each + Add&Norm + FFN).

Fine-tuning pretrained ViT

- Full: new class token + new MLP head, train on new dataset.
- **PEFT principle**: insert additional components that **favour (don't change) the original ViT computation**; freeze backbone, **fine-tune / optimise only the new components**.

Prompt-tuning: learnable prompts inserted among the token embeddings

Adapter (in the pointwise FFN): $f(X) = g(X) + \sigma(XW_{down})W_{up}$, $g(X) = \sigma(XW_1)W_2$ frozen

LoRA (on W_Q, W_K, W_V): $W_Q + B_Q A_Q$, W:[m,n], B:[m,r], A:[r,n], $r \ll m,n \rightarrow r(m+n)$ trainable vs mn

★ Week 10 key concepts (what the quiz tests)

- Self- and cross-attention both compute all tokens **in parallel**. Self-attention: Q, K, V from the source, captures **intra**-sequence dependencies. Cross-attention: Q from the target, K and V from the source, captures **inter**-sequence dependencies.
- **CNN drawbacks**: no global information, can't capture spatial relationships between local objects, locality-sensitive.
- ViT: token = **patch**, turned into an embedding by a linear projection; the class token is **learnable**; the MLP head sits on the class token at the final layer. Global information comes from **multi-head self-attention**, and the final class token summarises the image.
- ViT is naturally global and finds long-range dependencies, but needs **massive data**: it can't be trained directly on small datasets. It is **more** robust than CNNs.
- Swin: patches of [3, 4, 4] → [C, H/4, W/4]. **Patch merging**: concatenate 2×2 neighbours, then a linear layer → [2C, H/8, W/8] (half the size, double the depth).
- **W-MSA**: attention inside each local window independently, same output shape, faster, but tokens only meet tokens in their own window. **SW-MSA**: windows shifted right and down so tokens interact across windows; same output shape.
- **PEFT**: add components that keep the original ViT computation and train only them. Prompts go into the **token embeddings**, adapters into the **pointwise FFN**, LoRA into **W_Q, W_K, W_V**.

WEEK 11 · Generative models: GAN & Diffusion

Density estimation

- **Explicit**: learn $p_m(x) \approx p_d(x)$, e.g. GMM $p(x) = \Sigma_k \pi_k N(x|\mu_k,\Sigma_k)$ fitted with EM; fails on high-dim / structured data (images, text). **Implicit**: learn generator G: $z\sim p(z) \rightarrow \tilde{x} = G(z)$ that mimics data (GAN, VAE). Generative model's task: **generate realistic examples imitating the dataset from noise**.

GAN (Goodfellow 2014)

$\min_G \max_D J(G,D) = E_{x\sim p_d}[\log D(x)] + E_{z\sim p_z}[\log(1 - D(G(z)))]$

D (ascent): $\nabla_{\theta_d} (1/M)\Sigma[\log D(x^{(i)}) + \log(1 - D(G(z^{(i)})))]$

G (descent): $\nabla_{\theta_g} (1/M)\Sigma \log(1 - D(G(z^{(i)})))$ ⟶ in practice $\max_G E_z \log D(G(z))$ (stronger gradient when fakes look fake); k = 1 D-step per G-step

$D(x) = P(\text{real} \mid x)$ via sigmoid; $1 - D(x) = P(\text{fake} \mid x)$

$D^*(x) = p_d(x) / (p_d(x) + p_g(x)) = 1 / (1 + p_g/p_d)$ (from $\partial/\partial D$: $p_d/D - p_g/(1-D) = 0$)

$J(D^*,G) = JSD(p_d \| p_g) - 2\log 2 \rightarrow$ min at $p_g^* = p_d$; Nash equilibrium $D^*(x) = 0.5\ \forall x$

- Analogy: G = counterfeiter making fake money; D = police detecting it. Stop at Nash equilibrium (no player can improve).
- More overlap of p_g, p_d ⇒ D's job **harder**, accuracy → 50%, BCE loss **larger**. GAN ↔ JSD, f-GAN ↔ f-divergence, WGAN ↔ Wasserstein.
- **Mode collapse**: fakes cover only a few modes, miss others; source: nothing constrains G to cover all modes. Issues: mode collapse, unrealistic images on complex data (ImageNet), unstable training (min-max, no unique loss). Conditional GANs work better.
- **Inception score** (higher = better): $p(y|\tilde{x})$ confident (low entropy) & average $p(y) \approx$ uniform (diverse): IS ≈ $E_z[KL(p(y|G(z)) \| p(y))]$.
- **DCGAN**: strided conv in D, fractional-strided (transposed) conv in G, no pooling; BatchNorm in both; no FC hidden layers; ReLU in G (tanh output); LeakyReLU in D. Latent arithmetic works.

ConvTranspose2d out = $(in - 1)\cdot s - 2p + k +$ output_padding Conv2d out = $\lfloor (in + 2p - k)/s \rfloor + 1$

Example: Linear(100→7·7·256) → Unflatten(256,7,7) → ConvT(256→128, k5, s3, p2): $(7-1)\cdot3 - 4 + 5 = 19$ → (16,128,19,19)

Diffusion models (DDPM)

Forward: $q(x_t|x_{t-1}) = N(\sqrt{(1-\beta_t)}\, x_{t-1}, \beta_t I)$, $x_t = \sqrt{(1-\beta_t)}\, x_{t-1} + \sqrt{\beta_t}\,\varepsilon$

$\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{i\le t}(1-\beta_i)$, $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\,\varepsilon$, $\varepsilon \sim N(0,I)$; $\bar{\alpha}_t \rightarrow 0 \Rightarrow x_T \sim N(0,I)$

Reverse: $p_\theta(x_{t-1}|x_t) = N(\mu_\theta(x_t,t), \sigma_t^2 I)$; $q(x_{t-1}|x_t)$ intractable, $q(x_{t-1}|x_t,x_0)$ tractable

$\mu_\theta = (1/\sqrt{\alpha_t})(x_t - \beta_t/\sqrt{(1-\bar{\alpha}_t)}\cdot\varepsilon_\theta(x_t,t))$; $\sigma_t^2 = \beta_t$ or $(1-\bar{\alpha}_{t-1})\beta_t/(1-\bar{\alpha}_t)$

Loss: $\min E\,\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$ Sample: $x_T\sim N(0,I)$; $x_{t-1} = \mu_\theta + \sigma_t z$, down to x_0

- **Forward** = shrink + add noise to clean images (closed form, no learning). **Backward** = start from noise, denoise each step (U-Net).
- U-Net $\varepsilon_\theta(x_t,t)$: residual blocks + self-attention; inputs x_t and time embedding; **predicts the added noise ε**; crucial for denoising; does NOT do the forward pass.
- Noise schedule: linear (2nd half almost pure noise) vs cosine (noises more slowly).
- **Latent diffusion** (Stable Diffusion): encoder $\mathcal{E}$ / decoder $\mathcal{D}$, diffuse $z_0 = \mathcal{E}(x_0)$ in latent space (faster); condition via cross-attention: Q = $W_Q\varphi(z_t)$, K,V = $W\cdot\tau_\theta(y)$; loss $\|\varepsilon - \varepsilon_\theta(z_t, t, \tau_\theta(y))\|^2$.

★ Week 11 key concepts (what the quiz tests)

- A generative model's job: generate realistic examples that imitate the dataset, starting from noise.
- Noise goes into **G**, not D. D separates real from fake (high score for real, low for fake) and so measures the gap between p_g and p_d; G mimics real data. G = counterfeiter, D = police.
- Objective: $\min_G \max_D E\log D(x) + E\log(1-D(G(z)))$. Swapping the two logs (labelling real 0, fake 1) is **also valid**; $\max_G \min_D$ is the wrong order.
- Alternating updates: $\max_D J$, then for G either min E log(1−D(G(z))) (original) or max E log D(G(z)) (used in practice).
- When p_g and p_d overlap more, D's job gets **harder**, its accuracy approaches 50% and its BCE loss gets **larger**.
- D ends in a sigmoid; D(x) = P(real), 1−D(x) = P(fake). $D^* = p_d/(p_d+p_g) = 1/(1+p_g/p_d)$. At the Nash equilibrium $D^* = 0.5$ and $p_g = p_d$.
- **Mode collapse** = generated data covers only a few modes. Causes: the min-max problem can't be solved perfectly, and nothing forces G to cover all modes. Training issues: mode collapse, unrealistic images on complex data, unstable training.
- Transposed-conv output = (I−1)·S − 2P + K + output_padding: (7−1)·3 − 4 + 5 = 19.
- **Forward** process = shrink the image and add noise; **backward** = start from noise and denoise step by step. Correct forward formulas: $x_t = \sqrt{(1-\beta_t)}x_{t-1} + \sqrt{\beta_t}\varepsilon$ and $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{(1-\bar{\alpha}_t)}\varepsilon$ (wrong options swap x_0 and ε, or run t backwards).
- The U-Net is trained by **minimising** $\|\varepsilon_\theta(x_t,t) - \varepsilon\|^2$, predicts the added noise and does the denoising; it does **not** perform the forward process.